

MALAGA – MultidimensionAL Big DAta Analytics over Massive Graph DAta

Alfredo Cuzzocrea*
iDEA Lab
University of Calabria
Rende, Italy
& Dept. of Computer Science
University of Paris City
Paris, France
alfredo.cuzzocrea@unical.it

Mojtaba Hajian
iDEA Lab
University of Calabria
Rende, Italy
mhajian.idealab.unical@gmail.com

Abderrraouf Hafsaoui
iDEA Lab
University of Calabria
Rende, Italy
ahafsaoui.idealab.unical@gmail.com

Abstract—Focusing on the main research context represented by the issue of *supporting big data analytics over big graph data*, this paper introduces and experimentally assesses the framework MALAGA (*MultidimensionAL Big Data Analytics over Massive Graph DAta*). MALAGA incorporates several innovations, including *OLAP analysis of big graph data*, *columnar-OLAP methodologies*, and *Apache Hive extensions*. A comprehensive experimental assessment and analysis of the framework’s performance is finally presented and discussed, by significantly integrating the conceptual contributions of our research.

Keywords—*Big Data, Big Data Analytics, Multidimensional Big Data Analytics, Big Graph Data, Massive Graph Data*

I. INTRODUCTION

In today’s data-driven world, the ability to analyze and interpret vast amounts of information is crucial. *Big graph data analytics* (e.g., [1-3]) is a powerful approach that leverages graph theory to understand complex relationships within large datasets. Unlike traditional data models that focus on individual data points, *graph analytics* emphasizes the connections between these points, represented as nodes and edges in a graph.

Graphs are particularly effective for modeling real-world systems such as *social networks*, *biological networks*, and *transportation systems*. By analyzing these graphs, we can *uncover hidden patterns*, *detect anomalies*, and *gain insights into the structure and dynamics of the underlying data*. This approach is invaluable for applications ranging from *fraud detection* and *recommendation systems* to *network optimization* and *scientific research*.

The rise of big data technologies has made it possible to *handle and process massive graph datasets efficiently* (e.g., [4]). Tools like *Neo4j* (e.g., [5]) and *GraphX* (e.g., [6]) provide the necessary infrastructure to store, query, and analyze large-scale graphs. As a result, organizations can now harness the full potential of their data, driving innovation and informed decision-making. In summary, big graph data analytics is a transformative field that enables us to explore and understand the intricate web of relationships within large datasets, offering new opportunities for discovery and advancement across various domains.

When we consider algorithms that support analytics over massive big graph data, several key research problems arise [4]. First, *scalability* is one of the primary challenges. It consists in ensuring that algorithms can scale efficiently as the size of the graph increases. This involves optimizing both time and space complexity to handle billions of nodes and edges.

Following this, *parallelism and load balancing* is a possible solution. Indeed, effective parallel processing is crucial for handling large-scale graphs. Ensuring that the workload is evenly distributed across multiple processors or machines to avoid bottlenecks and maximize resource utilization is a significant challenge. However, this has some side effects. In fact, in distributed systems, *minimizing the communication overhead* between different nodes is essential. Contrary to this, excessive data transfer can slow down the processing and increase latency.

In addition to this, it must be considered that many real-world graphs are *dynamic*, with nodes and edges being added or removed over time. Developing algorithms that can efficiently handle such changes without needing to recompute everything from scratch is a complex problem [7]. Overall, efficiently *partitioning a graph into smaller subgraphs* can significantly improve processing times. However, finding optimal partitions is an *NP-complete problem*, making it a challenging area of research.

Optimizing queries to retrieve information quickly from large graphs is another critical area. This involves developing indexing and caching strategies that can handle complex queries efficiently [8]. As regards instead proper analytics aspects, ensuring that the data used in graph analytics is clean and well-integrated from various sources is essential for accurate analysis. This includes dealing with inconsistencies, missing data, and different data formats, thus leveraging *data integration and cleaning* research issues.

Summarizing, these challenges highlight the complexity and interdisciplinary nature of big graph data analytics, requiring advancements in algorithms, distributed computing, and data management techniques. In line with this research area, this paper proposes and experimentally assesses MALAGA, a framework for supporting *MultidimensionAL Big DAta Analytics over Massive Graph Data*. MALAGA is an innovative big data analytics framework specifically aiming at applying the *multidimensional big data analytics paradigm* [9-12] to big graph datasets. This involves in a series of technical and technological advancements that includes, among others: (i) *OLAP analysis* [13] (ii) *columnar-based OLAP*; (iii) *Apache Hive extensions*. All these contributions are described and presented in this paper.

The remaining part of this paper is structured as follows. Section II offers a comprehensive overview of state-of-the-art proposals related to big data analytics, particularly in the context of massive graph data. Section III describes the methodologies used in this research, including the architecture of the MALAGA framework and implementation details. Section IV presents the experimental setup, datasets and

* This research has been made in the context of the Excellence Chair in Big Data Management and Analytics at University of Paris City, Paris, France

performance evaluation. Finally, Section V provides conclusions and outlines future directions.

II. RELATED WORK

In this Section, we provide an overview of state-of-the-art proposals related to big data analytics over massive graph data.

[14] makes a dual contribution to *graph-oriented analytical processing*. Firstly, it introduces a novel approach for modeling *graph-based data warehouses* as well as proposing a method for data cube materialization by pre-computing aggregated nodes. They illustrate how standard OLAP queries can be executed on data warehouses stored in NoSQL graph-oriented databases. Using *Neo4j* and the *Cypher query language*, the implementation is compared with a relational data warehouse, by demonstrating that the graph-based OLAP implementation significantly outperforms the relational one in terms of query response time.

In [15], authors address the growing need for effective analysis of massive graph data in real-life applications by evaluating the performance of various *distributed graph computing systems*. While these systems are based on *vertex-centric model* [16] and have introduced techniques to overcome its limitations, there has been no comparative analysis to guide users in selecting the best system. Through extensive experiments on graphs with different characteristics and algorithms, this paper assesses the performance, scalability and effectiveness of these systems, which offers valuable insights for users and researchers by highlighting both strengths and limitations.

[17] focuses on addressing the challenge of finding *2-Edge Connected Components (2-ECCs)* in massive undirected graphs, which is crucial for various applications such as social network analysis, data mining, etc. Existing *single-node* algorithms are impractical for handling large graphs due to their limitations in processing time. To address these drawbacks, the paper proposes a distributed and parallel algorithm, named *BiECCA*, designed to efficiently process large-scale graphs using the *MapReduce* framework, as well as including a *complexity analysis* of the algorithm and outlines novel extensions to further enhance the approach.

In [18], authors introduce a novel graph OLAP framework that addresses the need for multi-dimensional and multi-level graph analysis. First, the paper develops a conceptual framework for data cubes on graphs, by defining dimensions and measures for the graph OLAP scenario and classifying OLAP operations into two categories (i.e., *informational* and *topological OLAP*). Second, it focuses on informational OLAP, demonstrating how graph data cubes can be efficiently materialized by calculating aggregated graphs, and includes both full and partial materialization with *iceberg* data cubes (e.g., [19]).

[20] introduces a framework for scalable *graph-based OLAP analytics* over large-scale heterogeneous and unstructured process data, named *big process graphs*, where traditional OLAP techniques struggle to analyze such data due to its complexity and the interactions among objects. The paper presents a model for *Process OLAP (P-OLAP)*, which defines key OLAP abstractions like process cubes, dimensions, and cells, tailored to the process context. Additionally, it introduces a *MapReduce-based graph-processing engine* to support scalable analytics over big

process graphs. The framework, integrated into the *ProcessAtlas platform*, enables efficient querying and exploration of large process data.

[21] provides a comprehensive overview of the current state-of-the-art in *distributed graph processing*, by focusing on distributed environments challenges (e.g., *parallelism*, *load balancing*, *communication overhead* and *bandwidth*). While many studies have focused on *programming models*, this research systematically analyzes the challenges of applying graph tasks in distributed settings. It surveys the existing solutions proposed in active literature and highlights the strategies used to address these issues.

In [22], authors introduce *TurboGraph++*, a *scalable and efficient graph analytics system* designed to process large-scale graphs using external memory. *TurboGraph++* bridges the gap between existing systems that prioritize efficiency at the risk of out-of-memory errors. Its key contributions include a new graph processing for *neighborhood analytics* (e.g., triangle counting and local clustering coefficient), and a *balanced partitioning scheme* in order to fully utilize CPU, disk, and network resources. The experimental evaluations demonstrate that *TurboGraph++* scales to large graphs while achieving good performance.

[23] proposes *GraphH*, a system to support *high-performance big graph analytics* in small clusters. Authors introduce a *two-stage graph partitioning scheme* to divide the input graph into partitions, which are processed in memory using a *Gather-Apply-Broadcast (GAB)* computation model. To minimize *disk I/O overhead*, *GraphH* employs an *edge cache mechanism*, while a hybrid communication strategy improves *inter-worker communication*. Extensive evaluations demonstrate that *GraphH* achieves significant performance improvements with respect to *in-memory systems*.

In [24], authors investigate the problem of capturing and querying provenance from high-performance graph data processing systems and propose a *batch-processing tool* called *Ariadne*. *Ariadne* addresses the challenge of managing vast *query data provenance* by offering a concise declarative query language that allows querying provenance information in real-time. The system leverages the formal semantics of a *datalog-based* language to identify classes of queries that can run concurrently with graph analytics.

[25] describes *I-HASTREAM*, a *density-based hierarchical clustering algorithm* over time-variant *big data streams* for big graph analytics purposes. *I-HASTREAM* addresses the increasing need for efficient knowledge extraction from data streams, which is crucial for a wide range of modern applications. Authors highlight a special case of *graph-shaped data streams*, which provide both structural and *content-oriented knowledge*. To support the analysis of such data, this research introduces big graph analytics engine architecture, where *I-HASTREAM* serves as the core component.

Finally, [26] presents *TgStore*, an efficient storage system designed for *large-scale time-evolving graphs*, which addresses the key challenges of *storage cost* and snapshot access speed. To tackle this, authors propose two innovative strategies, a *Snapshot-level Data Deduplication (SLDD)* technique that eliminates redundant vertices and edges across snapshots, and a *Structure-Changing Graph Representation (SCGR)* to enhance snapshot access speed.

III. THE MALAGA FRAMEWORK

In this Section, we describe the main approaches we adopt in our research to accelerate range query responses over OLAP data cubes, leveraging the capabilities of the MALAGA framework.

MALAGA is an *open-source, Hadoop-compliant Java-based framework* designed to support multidimensional big data analytics over massive graph data. Massive graph data arise in a plethora of big data application settings, ranging from *bioinformatics* to *smart cities*, from *social network analysis* to *intelligent transportation systems*, and so forth. In this so-delineated scenario, supporting big data analytics over massive graph data is an emerging research challenge in the big data research context, with also surprising presence of inspiring platforms and technologies coming from the industry. Looking at active literature, there exist several research proposals that focus the attention on the issue of supporting big data analytics over massive graph data (e.g., [10,11]), which confirms the popularity of the topics.

Multidimensional big data analytics (e.g., [9-12]) positions itself as a state-of-the-art big data analytics paradigm where the main innovation consists of embedding fortunate *multidimensional metaphors* [13] into the target big data analytics process. Basically, this paradigm pursues the idea of modeling both the target big dataset and the big data process itself in terms of *dimensions* (of analysis) and *measures* (of interest), according to the underlying big data analytics goals. This contributes to magnify the expressive power and to achieve a relevant gain into the discovery of actionable-knowledge insights.

MALAGA framework provides beneficial supporting of all the tasks of the multidimensional big data analytics procedure including (i) data connection/alimentation over the massive graph data source, even in multiple fashions; (ii) ETL (*Extraction, Transformation & Loading*) over the massive graph data source, even in multiple fashions; (iii) definition of the multidimensional data model over the graph data source, according to the underlying big data analytics goals; (iv) data partitioning/distribution of the so-generated big multidimensional data structures over the target commodity Cloud; (v) definition of the multidimensional big data analytics tasks over the target big multidimensional data structures, from conventional ones (e.g., drill-down, roll-up, pivoting, etc.) to advanced ones (e.g., user-defined aggregations, etc.); (vi) definition of ad-hoc interactive dashboards over big multidimensional data analytics. Fig. 1

illustrates the comprehensive architecture of the entire framework.

It should be considered that, from a proper research perspective, implementing and realizing MALAGA encompasses several relevant research challenges. Indeed, while some research proposals have investigated the problem of supporting conventional OLAP operations over graph data (e.g., [29]), the problem is still an open research issue, particularly when considered as related to the emerging big graph data context.

These open problems can be addressed by defining innovative techniques and algorithms that take into consideration the well-understood properties of big data, including volume, velocity, and variety, which become even worse when associated with graph-shaped data sources. These techniques and algorithms are the most important progress of the MALAGA framework. Last but not least, defining and implementing suitable case studies showing the capabilities of MALAGA in emerging big graph data applications and systems.

A. Hive-Supported Column-Based OLAP

The increasing demand for scalable and high-performance analytics has driven the evolution of the OLAP paradigm. After, column-based OLAP systems excel in handling large-scale data and complex analytical queries by storing and processing data at the *column level* rather than the traditional *row-based approach*. This approach enables more efficient data retrieval, by reducing overhead, and optimizes query execution, particularly in environments that prioritize *read-heavy* operations, such as data warehousing and *Business Intelligence (BI)*. By shifting from traditional row-based storage and query models to a columnar-based architecture, the system optimizes the processing of complex and multidimensional data, which is essential for modern data analytics tasks.

Fig. 1 shows an overview of MALAGA, which is based on the proposed *Hive-Supported Column-Based* methodology. At the core of the approach lies the columnar representation of OLAP data cubes, where big data cubes are partitioned into individual columns. This column-based approach is ideal for OLAP systems as it allows for more targeted and efficient data access. Instead of scanning entire rows of data, as required in traditional systems, this approach enables queries to retrieve only the necessary columns, significantly reducing the volume of data being read and processed.

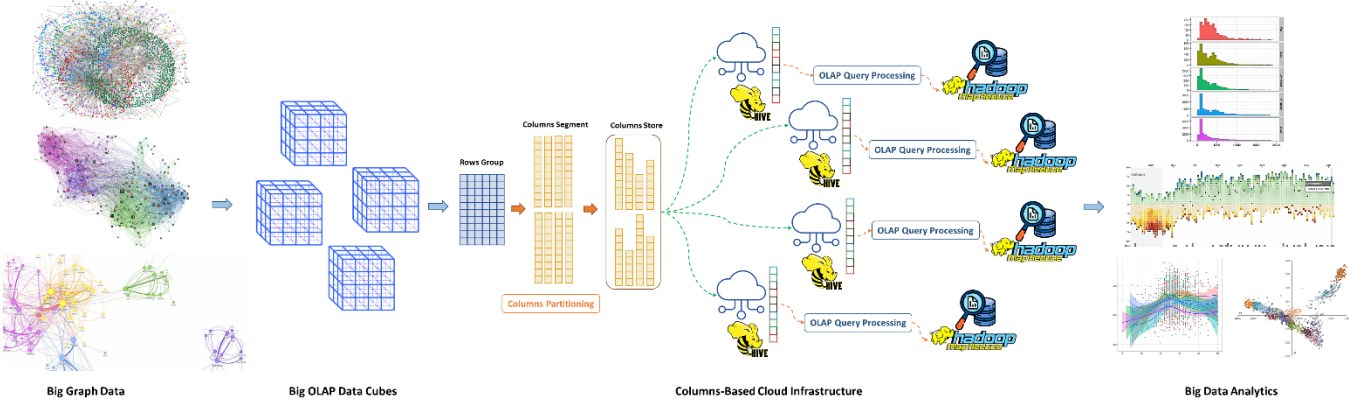


Fig. 1. Comprehensive Architecture of the MALAGA Framework

This selective data retrieval results in faster query execution times and a considerable reduction in computational overhead, especially when dealing with high-dimensional data. Furthermore, each column is distributed across multiple Cloud nodes, which ensures that MALAGA can efficiently scale as data grows.

In addition, Hive plays a critical role in this approach by acting as the intermediary layer that manages columnar data and facilitates efficient query execution. Hive robust data warehousing capabilities allow for the transformation of OLAP cubes into a well-structured, column-based format that is optimized for high-performance queries. In addition to this, MALAGA employs the *Distributed Grid File Index* (DGFIIndex) [30] over columns, to speed-up query performance.

Moreover, *parallel processing and distributed architecture* are essential to unlocking the full potential of column-based OLAP systems. By leveraging modern Cloud technologies, such as Hive and MapReduce, columnar data are distributed across Cloud nodes. Hive plays a pivotal role in performing efficient column projection, which means that instead of retrieving all columns, only the necessary ones for a specific query are processed. This selective access reduces both the volume of data transferred and the *computational overhead*, which optimizes query performance.

MapReduce jobs further enhance query processing by enabling distributed, parallel computations across multiple nodes. These jobs are particularly effective in handling aggregate queries, where summarization and data aggregation tasks are executed simultaneously across different columns stored on various Cloud nodes. This approach takes advantage of the Cloud scalability, ensuring that query response times remain efficient even as data volumes grow.

The benefits of column-based OLAP systems are most evident in scenarios involving large datasets with multidimensional data cubes. OLAP cubes, often characterized by their multidimensional structure, can be partitioned into distinct columns for finer-grained data access and manipulation. This granularity enables faster, more targeted queries and contributes to the overall scalability of the MALAGA framework. This column-based approach includes key benefits and advantages such as:

- *Improved Query Performance*: by fetching only the necessary columns for a query data transfer is minimized, reducing the time to process complex analytical queries.
- *Efficient Data Compression*: columnar storage leads to higher compression rates, which reduces storage costs and improves read performance.
- *Parallel and Distributed Processing*: Distributed Cloud architectures, combined with parallel processing frameworks like MapReduce, allow MALAGA to scale horizontally and handle massive datasets efficiently.

In summary, this Hive-supported column-based OLAP approach offers a powerful, scalable, and efficient solution for managing and analyzing large-scale, multidimensional datasets within Cloud environments. By leveraging Hive advanced capabilities, MapReduce for parallel query execution, and the scalability of Cloud infrastructure,

MALAGA is able to deliver high-performance analytics for even the most complex OLAP queries.

B. Distributed Grid File Index

In MALAGA, DGFIIndex is employed to efficiently manage *Multidimensional Range Queries* (MDRQs) [31] within a big data processing environment. This index structure is applied to OLAP-derived columns and it has been selected due to its superior performance in handling large datasets, such as those generated by smart grid systems where substantial amounts of meter data are collected continuously [30].

The DGFIIndex works by dividing the data space into cubes using a grid file technique. Unlike traditional Hive indexes, which store all combinations of multiple dimensions, the DGFIIndex only stores the bucket's information. This approach results in a significantly smaller index size and faster query processing, particularly for aggregation queries. Additionally, the DGFIIndex allows for the pre-computation of user-defined aggregations for each bucket, which minimizes the need for extensive data access during query execution.

The DGFIIndex is constructed through a MapReduce job that reorganizes the data into slices (buckets) based on a pre-defined splitting policy. These slices are then stored as key-value pairs in a distributed key-value store, such as *HBase*, which enhances the speed of index retrieval. The splitting policy is defined by specifying the minimum value and interval size for each index dimension, allowing for efficient cube-based data partitioning. This method also stores pre-computed values for each bucket, which further optimizes the retrieval process during aggregation queries.

The decision to utilize the DGFIIndex was made because of its significant performance improvements compared to traditional indexing methods in Hive, such as the Compact Index, Bitmap Index, and Aggregate Index. These traditional methods are less effective at handling MDRQs [30]. In contrast, the DGFIIndex's smaller size, faster access capabilities, and ability to filter unrelated splits and slices make it an ideal choice for large-scale, high-throughput data environments.

Here, we present the DGFIIndex approach in more detail. DGFIIndex divides the data space into *Grid File Units* (GFUs) using a grid-like structure, enabling efficient access to data by splitting the space into buckets or regions. These units are stored as key-value pairs in a distributed system such as *HBase*, allowing fast lookups and selective retrieval during query execution. The main goal of DGFIIndex is to reduce the size of the index and avoid full-table scans by filtering irrelevant data early in the process.

1) *Grid File Unit Definition*: The data space is divided into GFUs, where each GFU corresponds to a partition in the data based on certain dimensional values. Each GFU contains:

- *GFUKey*: The key that identifies the minimum value of a specific dimension's grid unit in the data space.
- *GFUValue*: The value that consists of two parts: (i) the metadata (*header*) containing *pre-computed aggregates* and (ii) the *offset to the physical location* of the slice (data) in the storage.

This process is formalized as follows. The splitting policy defines the granularity of the grid by specifying the given *number of buckets* (N) and computing *interval size* (ΔX_d) as follows:

$$\Delta X_d = \frac{\max(X_d) - \min(X_d)}{N} \quad (1)$$

where X_d is a set of all values in dimension d . Then, the lower and upper boundaries of each bucket's values are obtained as (l_i, u_i) , $i = 1, \dots, N$. After defining the boundaries of the buckets, the values of column data are reorganized to GFU files based on conditions derived from their boundary (l_i, u_i) , which later will be used in query processing.

Additionally, given D dimensions in the data space (e.g., dimensions like *date*, *location*, *price*, etc.), the GFUKey for a specific bucket is identified as the *lower limit of boundary* (l_i) of the bucket (see Tab. I).

TABLE I. DGFINDEX TABLE

GFU Key	Header Sum	File Name
2	1,080	Bucket 1
27.33	850	Bucket 2
52.66	1,400	Bucket 3

Eventually, this procedure maps data points into discrete grid units by dividing each dimension based on the user-defined number of buckets. Fig. 2 illustrates a comprehensive example of DGFIndex operation on a column-based data.

As shown in Fig. 2, the dimension and measure column data are split and reorganized according to the number of buckets defined by user, and the range of values in dimension column data. This procedure results in GFUs as data buckets which later will be used in efficient query answering. After creating the GFUs, it is necessary to create a DGFIndex table, which is used in the querying phase to find the boundary, and *inner buckets* as well as the pre-computed aggregation value.

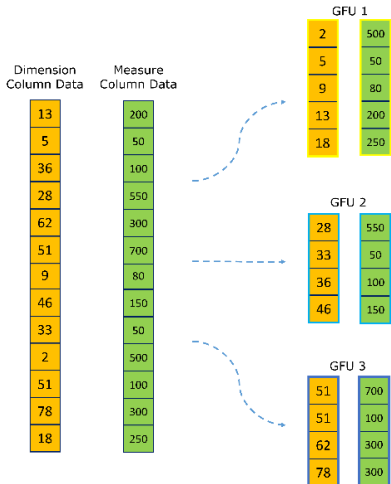


Fig. 2. DGFIndex Operation on a Column-Based Data

2) *Index Construction Algorithm (Using MapReduce)*: The DGFIndex is constructed using the MapReduce framework, which reorganizes the data into slices based on the GFUKey values. The construction process follows these

steps: (i) *Map Phase*: the Map function computes the GFUKey for each record and groups data points belonging to the same GFU into a slice. (ii) *Reduce Phase*: in the Reduce function, all records with the same GFUKey are grouped and stored as a slice. The slices are stored in the *Hadoop Distributed File System* (HDFS), and the index is stored in HBase for fast retrieval.

3) *Query Execution Using DGFIndex*: Once the DGFIndex is constructed, query execution becomes highly efficient by using the index to filter irrelevant data. When a query is issued, Hive retrieves the relevant GFUs by comparing the query predicates to the index. The query engine calculates the GFUKey range for each dimension based on the query's filtering conditions, thus, the relevant GFUs are those with GFUKeys within the ranges. Next, Hive filters out irrelevant splits in HDFS using the offsets stored in the GFUValues. This ensures that only the data slices relevant to the query are processed. For aggregation queries, such as the sum of the price, DGFIndex can skip reading the actual data for certain slices if the pre-computed aggregates (such as SUM, COUNT, or AVG) stored in the GFUValue are sufficient to satisfy the query. For example, if an entire slice falls within the query's range, Hive can use the pre-computed aggregate instead of scanning the actual data, reducing I/O and speeding up the query.

For slices in an aggregation range query, the aggregation result is computed as follows:

$$\text{Query Result} = \sum_{S \in R} (A_{pre} + A_{partial}) \quad (2)$$

where, S is slices involved in the range query R , A_{pre} is pre-computed aggregate value for the slices entirely involved in the range query, and $A_{partial}$ is aggregate value should be computed for the slices partially involved in the range query, also called *boundary slices*.

4) *Advantages of DGFIndex*: The contribution of DGFIndex presents several benefits to MALAGA that are as follows: (i) *Compact Index Size*: DGFIndex stores only the relevant slices and their locations, resulting in a smaller index size compared to traditional indexes. (ii) *Efficient Query Execution*: By using pre-computed aggregates and fast lookups in HBase, DGFIndex enables faster query execution. (iii) *Scalability*: DGFIndex is scalable for high-cardinality dimensions, making it suitable for large datasets in distributed environments. (iv) *Compatibility with Columnar Storage*: DGFIndex can be integrated with columnar formats like ORC or RCFile, improving performance by reducing I/O for columnar reads.

IV. EXPERIMENTAL RESULTS

In this Section, we present our experimental assessment of the proposed MALAGA framework. In MALAGA, as mentioned in previous Sections, we explore the utilization of graph data structures to enhance query processing efficiency, and to finally test big data management and analytics tools over massive graph data. Graph data structures offer a flexible and powerful way to model complex relationships between data points, which can be particularly beneficial in MDRQs

over large graph datasets, seeing MDRQs as baseline operators for a plethora of big data analytics procedures.

Our final goal is that of comparing the performance of MALAGA with traditional Cloud-based OLAP systems, as to appreciate the technological speed-up ensured by MALAGA over traditional architectures, especially due to the exploitation of the Hive-based solution. Therefore, we engineered an experimental testing solution based on which, starting from the input multidimensional-dataset / data-cube, we create a *graph-based version of the data*, by simply (1) modelling each node with the data of the corresponding tuple, and (2) modelling each edge with the integrity constraint existing among the linked tuples. Fig. 3 shows an example of this procedure. Here, a big OLAP view extracted from a medical dataset shows a few example tuples (left). In the right part, the graph construction related to the tuple with ID=2 and ID=7 is shown. By iterating this simple yet effective procedure, we can achieve very large big graph datasets starting from a variety of input multidimensional-datasets / data-cubes. Another strong consideration supporting our experimental dataset engineering approach is represented by the evidence stating that relational datasets are easily to manage, maintain and process, in addition to their “stable” data distributions. Therefore, the corresponding graph datasets inherit the same nice properties as well, allowing us to achieve a more solid and reliable experimental assessment and analysis (e.g., [32]).

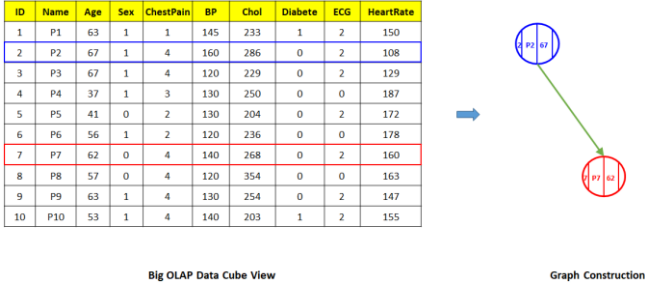


Fig. 3. Graph Construction Procedure from Relational Datasets

By integrating the graph data structure into MALAGA, we enhance the capability to process complex MDRQs. The graph-based querying complements the columnar storage and indexing methods by providing an additional layer of data organization that captures relationships between data points. The overall architecture includes the following steps:

- **Data Loading and Graph Construction:** Data is loaded from the OLAP cube and used to construct the graph, as described above.
- **Query Definition:** Users define queries with conditions over multiple dimensions.
- **Graph Traversal and Subgraph Extraction:** we perform graph traversal to extract subgraphs containing nodes that satisfy the query conditions.
- **Result Aggregation:** the extracted subgraph represents the query result, and further aggregation or analysis can be performed as needed.

In our experimental study, we conducted several experimental tests over *large-scale real-life datasets* in order to evaluate the performance and capabilities of our framework. We describe these datasets in Section A.

A. Dataset Description

Here, we describe datasets used in our experimental campaign. Specifically, we used the *TPC-H*, *Multidimensional Poverty Measures*, *Insurance Claim Analysis—Demographic and Health Factors*, and *Simulacrum* benchmark datasets.

1) **TPC-H:** TPC-H dataset [33] is a decision support benchmark designed to simulate a real-world data warehousing environment. The dataset models a business scenario where various parts of the business operations, such as *suppliers*, *customers*, and *orders*, are tracked. It allows testing of a wide range of query types to measure performance under a heavy load and complex analytical processing.

The TPC-H dataset consists of multiple relational tables that represent different aspects of a company's sales process. Each table is designed to capture specific information, allowing for the simulation of complex queries involving joins, aggregations, and filtering. The schema is normalized and consists of eight main tables. The schema follows a relational model with *foreign key relationships* that define how tables are related to each other. For example, the *Customer* table is linked to the *Orders* table via the *C_CUSTKEY*, and the *Orders* table is linked to the *LineItem* table via *O_ORDERKEY*. Fig. 4 shows the schema of the TPC-H dataset.

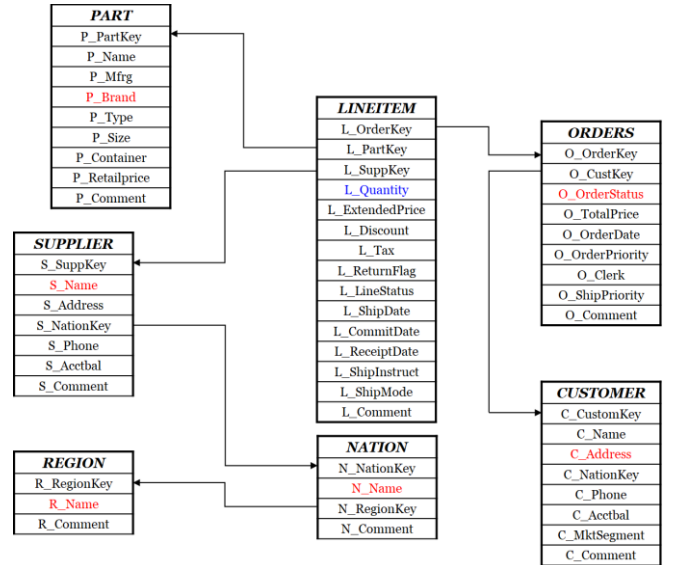


Fig. 4. Schema of the TPC-H Dataset

2) **Simulacrum:** *Simulacrum* dataset [34] is a synthetic dataset designed to emulate the cancer data held by the *National Cancer Registration and Analysis Service* (NCRAS) in England. Developed by *Health Data Insight* (HDI), it provides a safe and confidential way for researchers to explore cancer data structures, test hypotheses, and develop analytical models without accessing real patient information.

The dataset mirrors the complexity and structure of actual cancer data, including variables related to patient demographics, cancer diagnoses, treatment pathways, and outcomes. While all data is artificially generated and contains no real patient information, the *Simulacrum* maintains realistic

distributions and relationships between variables, allowing for meaningful analysis and model development. The dataset allows researchers to explore vital aspects such as the *age*, *gender*, and *cancer staging of synthetic patients*, and even conduct *survival analyses* based on their simulated vital status. Additionally, it facilitates studies on the genetic mutations found in various cancer types. Fig. 5 shows the schema of the *Simulacrum* database.

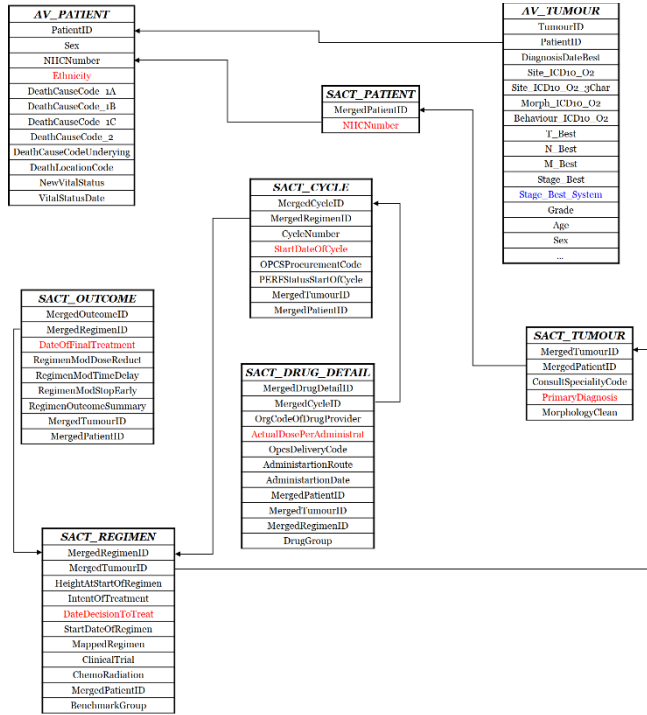


Fig. 5. Schema of the Simulacrum Dataset

3) *Multidimensional Poverty Measures*: The *Multidimensional Poverty Measures* dataset [35] provides a comprehensive analysis of poverty by incorporating multiple dimensions beyond traditional income-based metrics. Recognizing that poverty is a multifaceted phenomenon, this dataset includes indicators such as *health status*, *educational attainment*, *living standards*, *employment quality*, and access to essential services like clean water and electricity. By doing so, it captures the complex reality of poverty experienced by individuals and households across different countries.

This dataset is based on the use of the *Alkire-Foster (AF)* method for calculating the *Multidimensional Poverty Index (MPI)*. The AF method identifies overlapping deprivations across various dimensions, allowing for a nuanced assessment of who is poor and how they are poor. The MPI is calculated separately for urban and rural areas within each country, providing insights into spatial disparities in poverty levels. Key variables in the dataset include *ISO* (country identifiers), *Country* (country names), *MPI Urban/Rural* (the MPI values for urban and rural areas), *Headcount Ratio Urban/Rural* (the proportion of the population identified as multidimensionally poor in urban and rural areas), *Intensity of Deprivation Urban/Rural* (the average proportion of deprivations experienced by the poor in urban and rural areas).

This dataset is valuable for researchers and policymakers aiming to design targeted poverty alleviation programs. By highlighting the specific deprivations faced by different

populations, it facilitates the formulation of interventions that address the root causes of poverty. Additionally, the dataset's international scope allows for cross-country comparisons and monitoring of global poverty trends. Fig. 6 shows the schema of the *Multidimensional Poverty Measures* database.

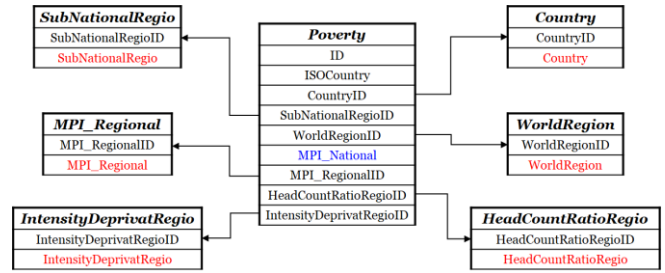


Fig. 6. Schema of the Multidimensional Poverty Measures Dataset

4) *Insurance Claim Analysis—Demographic and Health Factors*: The *Insurance Claim Analysis* dataset [36] offers detailed information on insurance claims, focusing on how demographic and health factors influence the risk and severity of claims. It includes variables such as *patient age*, *gender*, *Body Mass Index (BMI)*, *blood pressure levels*, *diabetic status*, *number of children*, *smoking status*, and so forth. The dataset also records the amount of each insurance claim, providing a direct link between personal attributes and financial outcomes.

With over 1,300 entries, this dataset enables comprehensive statistical analyses and the development of predictive models. Researchers can explore correlations between health metrics (e.g., *BMI*, *blood pressure*) and claim amounts, identify high-risk demographic groups, and assess the impact of lifestyle choices like smoking on insurance costs. The inclusion of regional data allows for geographic comparisons, which can inform market segmentation and targeted policy interventions.

Key variables include *PatientID* (unique identifier for each patient), *Age* and *Gender* (demographic information), *BMI* and *Blood Pressure* (health indicators), *Diabetic* and *Smoker* (boolean indicators of health-related behaviors), *Children* (number of dependents), *Region* (geographic location), *Claim* (insurance claim amount).

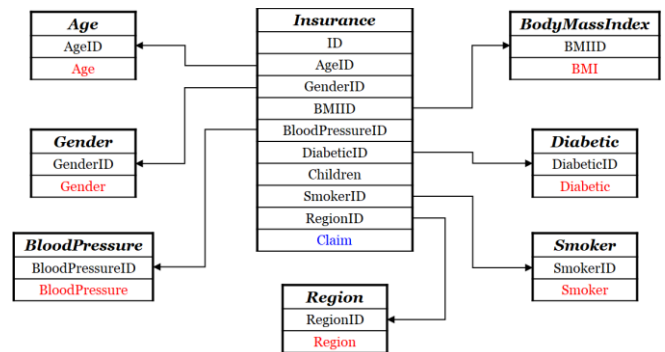


Fig. 7. Schema of the Insurance Claim Analysis Dataset

This dataset is instrumental for insurance companies in refining underwriting practices and developing risk assessment models. By understanding which factors are most predictive of high claim amounts, insurers can tailor premiums and coverage options accordingly. Public health officials may also use this data to identify populations at

greater health risks, enabling proactive health interventions. Fig. 7 shows the schema of the *Insurance Claim Analysis* database.

B. Results and Discussion

In this Section, we analyze the performance of our framework in order to assess its efficiency and effectiveness. For this, we investigate its performance against the previously described datasets.

In order to assess the efficacy of our framework, we compare the performance of MALAGA framework, with respect to the approach using *SQL Server Analysis Server* (SSAS) still within a Cloud environment. To accomplish this, we measured the *execution time* of the queries over the OLAP data cubes with respect to the varying number of dimensions N using the selected datasets, i.e., TPC-H, *Simulacrum*, *Multidimensional Poverty Measures* and *Insurance Claim Analysis*, according to the graph dataset generation model described above.

Fig. 7 shows the query execution time comparison as the number of dimensions in the data cube increases for the case of the TPC-H dataset.

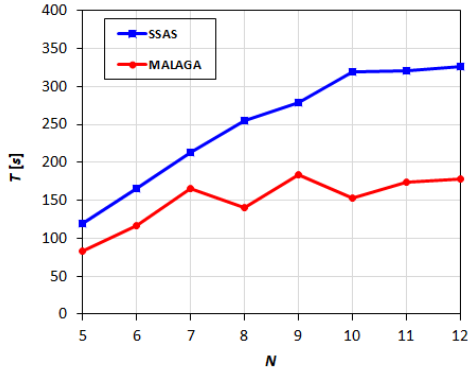


Fig. 8. Query Execution Time for TPC-H Dataset

As shown in Fig. 7, the query execution time increases significantly for SSAS approach as more dimensions are added. By contrast, MALAGA demonstrates a much slower increase in execution time.

Fig. 8 shows the query execution time comparison for the case of the *Simulacrum* dataset.

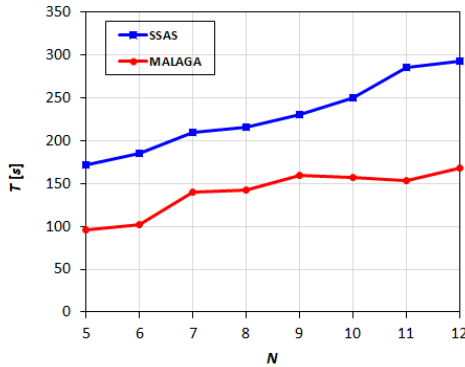


Fig. 9. Query Execution Time for Simulacrum Dataset

Similar trends are observed with the *Simulacrum* dataset, which involves complex healthcare data. The approach SSAS struggles with high-dimensional queries, leading to a sharp

rise in execution time as dimensions increase. In contrast, the MALAGA significantly reduces the performance burden, achieving a faster execution time.

On the other hand, Fig. 9 shows the query execution time comparison between MALAGA and SSAS for the case of *Multidimensional Poverty Measures* dataset.

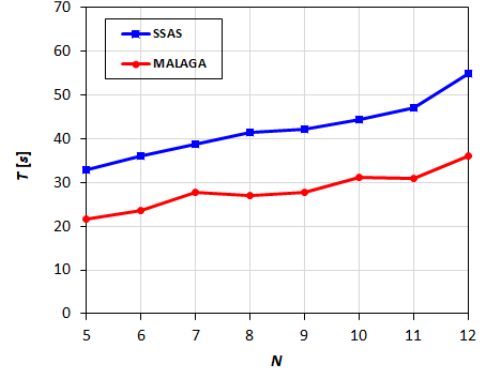


Fig. 10. Query Execution Time for Multidimensional Poverty Measures Dataset

As can be seen from Fig. 9, MALAGA proves particularly beneficial as the dimensionality increases in this dataset, which involves multiple indicators such as health, education, and living standards. Contrarily, SSAS approach, query execution time rises sharply with each additional dimension, which makes this approach impractical for high-dimensional data cubes.

Finally, Fig. 10 shows the query execution time comparison for the case of *Insurance Claim Analysis* dataset.

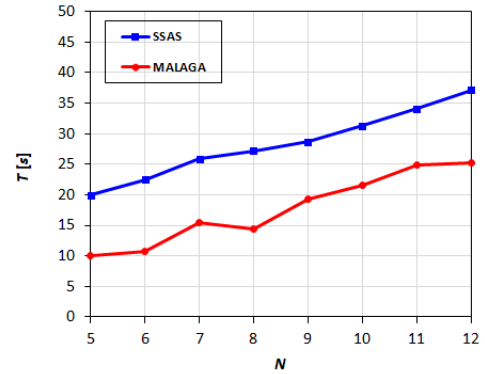


Fig. 11. Query Execution Time for Insurance Claim Analysis dataset

In summary, this experimental evaluation clearly shows that MALAGA outperforms the SSAS approach in terms of query execution time, especially as the dimensionality of the data cube increases. As a result, in high-dimensional data environments, the SSAS approach quickly becomes impractical. By contrast, the MALAGA efficient partitioning and aggregation capabilities enable it to handle complex, multidimensional queries with far greater efficiency, making it the preferred solution for big graph data analytics in environments where scalability and performance are crucial.

V. CONCLUSIONS AND FUTURE WORK

In this paper, we have proposed and experimentally assessed MALAGA framework, an innovative framework for enhancing multidimensional big data analytics over massive

graph data. The proposed framework, when evaluated across various real-life and benchmark datasets, demonstrates significant improvements in query execution time, particularly as the dimensionality of data cubes increases. Our experimental results show that the MALAGA outperforms Cloud-based SSAS approaches, which makes it a highly scalable and practical solution for high-dimensional big data environments, particularly in the context of massive graph data.

Future work will primarily focus on enhancing our framework to better address and deal with research challenges driven by *emerging big data trends* (e.g., [37-42]).

ACKNOWLEDGMENT

This research is supported by Tecnoter s.r.l., Italy.

The results of this research are in the context of the ICSC National Research Centre for High Performance Computing, Big Data and Quantum Computing within the NextGenerationEU program (Project Code: PNRR CN00000013).

REFERENCES

- [1] P. Sun, Y. Wen, T. N. B. Duong, X. Xiao, "GraphMP: I/O-Efficient Big Graph Analytics on a Single Commodity Machine". *IEEE Transactions on Big Data* 6(4), pp. 816-829, 2020
- [2] S. M. S. Hasan, D. Schmidt, R. Kannan, N. Imam, "A Scalable Graph Analytics Framework for Programming with Big Data in R (pbdR)". In: *IEEE International Conference on Big Data*, pp. 4783-4792, 2019
- [3] F. S. C. Tseng, A. Y. H. Chou, "Temporal Graph Warehousing for Big Data Analytics". In: *26th Pacific Asia Conference on Information Systems*, art. 305, 2022
- [4] L. Meng, Y. Shao, L. Yuan, L. Lai, P. Cheng, X. Li, W. Yu, W. Zhang, X. Lin, J. Zhou, "A Survey of Distributed Graph Algorithms on Massive Graphs". *CoRR abs/2404.06037*, 2024
- [5] Neo4J, *Neo4J Graph Database & Analytics*. Available at <https://neo4j.com/>
- [6] Apache Spark, *GraphX*. Available at <https://spark.apache.org/graphx/>
- [7] W.Y.H. Adoni, T. Nahhal, M. Krichen, A. El byed, I. Assayad, "DHPV: A Distributed Algorithm for Large-Scale Graph Partitioning". *Journal of Big Data* 7(1), pp. 1-25, 2020
- [8] Y. Bai, X. Lv, J. Li, L. Hou, "Answering Complex Logical Queries on Knowledge Graphs via Query Computation Tree Optimization". In: *International Conference on Machine Learning*, pp. 1472-1491, 2023
- [9] A. Cuzzocrea, "Innovative Paradigms for Supporting Privacy-Preserving Multidimensional Big Healthcare Data Management and Analytics: The Case of the EU H2020 QUALITOP Research Project". In: *4th International Workshop on Semantic Web Meets Health Data Management co-located with 20th International Semantic Web Conference*, pp. 1-7, 2021
- [10] A. Cuzzocrea, P.G. Bringas, "CORE-BCD-mAI: A Composite Framework for Representing, Querying, and Analyzing Big Clinical Data by Means of Multidimensional AI Tools". In: *17th International Conference on Hybrid Artificial Intelligent Systems*, pp. 175-185, 2022
- [11] A. Cuzzocrea, "Multidimensional Big Data Analytics over Big Web Knowledge Bases: Models, Issues, Research Trends, and a Reference Architecture". In: *8th IEEE International Conference on Multimedia Big Data*, pp. 1-6, 2022
- [12] A. Cuzzocrea, "A Reference Architecture for Supporting Multidimensional Big Data Analytics over Big Web Knowledge Bases: Definitions, Implementation, Case Studies". *International Journal of Semantic Computing* 17(4), pp. 545-568, 2023
- [13] J. Gray, S. Chaudhuri, A. Bosworth, A. Layman, D. Reichart, M. Venkatrao, F. Pellow, H. Pirahesh, "Data Cube: A Relational Aggregation Operator Generalizing Group-by, Cross-Tab, and Sub Totals". *Data Mining and Knowledge Discovery* 1(1), pp. 29-53, 1997
- [14] A. Khalil, M. Bellaissaoui, "A Graph-Oriented Framework for Online Analytical Processing". *International Journal of Advanced Computer Science and Applications* 13(5), pp. 547-555, 2022
- [15] Y. Lu, J. Cheng, D. Yan, H. Wu, "Large-Scale Distributed Graph Computing Systems: An Experimental Evaluation". *Proceedings of the VLDB Endowment* 8(3), pp. 281-292, 2014
- [16] Y. Tian, A. Balmin, S.A. Corsten, S. Tatikonda, J. McPherson, "From Think Like a Vertex to Think Like a Graph". *Proceedings of the VLDB Endowment* 7(3), pp. 193-204, 2013
- [17] D. Dhiphale, "MapReduce for Graphs Processing: New Big Data Algorithm for 2-Edge Connected Components and Future Ideas". *IEEE Access* 11, pp. 54986-55001, 2023
- [18] C. Chen, X. Yan, F. Zhu, J. Han, P.S. Yu, "Graph OLAP: Towards Online Analytical Processing on Graphs". In: *8th IEEE International Conference on Data Mining*, pp. 103-112, 2008
- [19] A. Cuzzocrea, F. Furfaro, G.M. Mazzeo, "A Probabilistic Approach for Computing Approximate Iceberg Cubes". In: *19th International Conference on Database and Expert Systems Applications*, pp. 348-361, 2008
- [20] A. Beheshti, B. Benatallah, H.R. Motahari-Nezhad, "Scalable Graph-based OLAP Analytics over Process Execution Data". *Distributed and Parallel Databases* 34(3), pp. 379-423, 2016
- [21] L. Meng, Y. Shao, L. Yuan, L. Lai, P. Cheng, X. Li, W. Yu, W. Zhang, X. Lin, J. Zhou, "A Survey of Distributed Graph Algorithms on Massive Graphs". *ACM Computing Surveys* 57(2), pp. 1-39, 2024.
- [22] S. Ko, W. Han, "TurboGraph++: A Scalable and Fast Graph Analytics System". In: *ACM International Conference on Management of Data*, pp. 395-410, 2018
- [23] P. Sun, Y. Wen, T.N.B. Duong, X. Xiao, "GraphH: High Performance Big Graph Analytics in Small Clusters". In: *IEEE International Conference on Cluster Computing*, pp. 256-266, 2017
- [24] V. Papavasileiou, K. Yocum, A. Deutsch, "Ariadne: Online Provenance for Big Graph Analytics". In: *ACM International Conference on Management of Data*, pp. 521-536, 2019
- [25] M. Hassani, P. Spaus, A. Cuzzocrea, T. Seidl, "I-HASTREAM: Density-Based Hierarchical Clustering of Big Data Streams and Its Application to Big Graph Analytics Tools". In: *16th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*, pp. 656-665, 2016
- [26] Y. Cheng, Y. Ma, H. Jiang, L. Zeng, F. Wang, X. Xu, Y. Wu, "TgStore: An Efficient Storage System for Large Time-Evolving Graphs". *IEEE Transactions on Big Data* 10(2), pp. 158-173, 2024
- [27] D. Yan, Y. Bu, Y. Tian, A. Deshpande, "Big Graph Analytics Platforms". *Foundations and Trends Databases* 7(1-2), pp. 1-195, 2017
- [28] D. Yan, Y. Tian, J. Cheng, "Systems for Big Graph Analytics". Springer Briefs in Computer Science, Springer, 2017
- [29] C. Chen, X. Yan, F. Zhu, J. Han, P.S. Yu, "Graph OLAP: A Multi-dimensional Framework for Graph Data Analysis". *Knowledge and Information Systems* 21(1), pp. 41-63, 2009
- [30] Y. Liu, S. Hu, T. Rabl, W. Liu, H. Jacobsen, K. Wu, J. Chen, J. Li, "DGFIndex for Smart Grid: Enhancing Hive with a Cost-Effective Multidimensional Range Index". *Proceedings of the VLDB Endowment* 7(13), pp. 1496-1507, 2014
- [31] A. Cuzzocrea, "Improving range-sum query evaluation on data cubes via polynomial approximation". *Data & Knowledge Engineering* 56(2), pp. 85-121, 2006
- [32] Y. Lu, D. Yang, P. Wang, P. Rosso, P. Cudré-Mauroux, "Schema-Aware Hyper-Relational Knowledge Graph Embeddings for Link Prediction". *IEEE Transactions on Knowledge and Data Engineering* 36(6), pp. 2614-2628, 2024
- [33] Transaction Processing Council, *TPC Benchmark H*. Available at <http://www.tpc.org/tpch/>
- [34] National Cancer Registration and Analysis Service, *Health Data Insight: The Simulacrum*. Available at <https://www.cancerdata.nhs.uk/simulacrum>
- [35] S. Alkire, M.E. Santos, *Multidimensional Poverty Measures*. Available at <https://www.kaggle.com/datasets/ophi/mpi>
- [36] S.K. Shukla, *Insurance Claim Analysis: Demographic and Health*. Available at <https://www.kaggle.com/datasets/thedevastator/insurance-claim-analysis-demographic-and-health>
- [37] A. Cuzzocrea, "CAMS: OLAPing Multidimensional Data Streams Efficiently". In: *11th International Conference on Data Warehousing and Knowledge Discovery*, pp. 48-62, 2009

- [38] B. Yu, A. Cuzzocrea, D.H. Jeong, S. Maydebura, "On Managing Very Large Sensor-Network Data Using Bigtable". In: *12th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing*, pp. 918-922, 2012
- [39] A. Cuzzocrea, R. Moussa, G. Xu, "OLAP*: Effectively and Efficiently Supporting Parallel OLAP over Big Data". In: *3rd International Conference on Model and Data Engineering*, pp. 38-49, 2013
- [40] A. Cuzzocrea, F. Furfaro, G.M. Mazzeo, D. Saccà, "A Grid Framework for Approximate Aggregate Query Answering on Summarized Sensor Network Readings". In: *OTM Confederated International Workshops*, pp. 144-153, 2004
- [41] Y. Yang, X. Yu, M. Serafini, A. Aboulmaga, M. Stonebraker, "Enhancing Computation Pushdown for Cloud OLAP Databases". *CoRR abs/2312.15405*, 2023
- [42] T. Wu, R. Gu, Y. Li, H. Ma, Y. Chen, Y. Zhu, X. Yu, T. Xu, Y. Huang, "Raven: Benchmarking Monetary Expense and Query Efficiency of OLAP Engines on the Cloud". In: *28th International Conference on Database Systems for Advanced Applications*, pp. 593-605, 2023